Part 3 Notes

Implementation notes, assumptions, and practical edge cases for the low-stock alerts endpoint.

Project	Inventory Management System for B2B SaaS
Section	Part 3 - API Notes and Assumptions
Endpoint	GET /api/companies/{company_id}/alerts/low-stock

The endpoint is expected at http://localhost:3000/api/companies/:company_id/alerts/low-stock. Authentication is assumed, but the current middleware only checks that a header is present, so proper JWT verification should be added before release.

Response

```json
{
  "alerts": [
    {
      "product_id": 1,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 10,
      "warehouse_name": "Main",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "daily_velocity": 0.43,
      "supplier": {
        "id": 5,
        "name": "Supplier Corp",
        "contact_email": "orders@supplier.com",
        "lead_time_days": 7,
        "minimum_order_quantity": 10,
        "unit_cost": 25.50
      }
    }
  ],
  "total_alerts": 42,
  "company_id": 1,
  "generated_at": "2026-04-08T16:45:23.123Z"
}
```

The main decisions I made

- **Recent sales activity:** I went with 30 days. The spec says "recent" but doesn't define it. 30 days is a common default for inventory velocity calculations and gives enough data points to make a meaningful average. This should probably be a configurable setting per company down the line.
- **Stockout estimate:** current_stock / avg_daily_sales, where avg daily sales = total units sold

in the last 30 days / 30.
It's a simple linear projection and won't account for seasonal spikes, but it's honest about what it is. A fancier version could use a weighted moving average (recent days count more than older ones), but I'd want to validate whether the product team even wants that complexity before building it.
Returns `null` if we can't compute it. Showing a fake number is worse than showing nothing.

- **Warehouse-level alerts:** A product might be adequately stocked in one warehouse and critically low in another. Collapsing those into one alert would hide the location-specific problem. The frontend can always aggregate if they want a product-level summary view.
- **Bundle behavior:** Bundles don't have their own physical stock — their components do. Alerting on bundle stock doesn't make sense with this model. Component alerts will fire instead. If the product team wants bundle-level alerts calculated from component availability, that's a separate piece of work.

Edge cases already considered

- Invalid company_id should return 400 status code instead coded of reaching the query layer.
- A missing company should return 404 status code instead of an empty success response.
- Products without suppliers should return supplier as null, not an empty object.
- If average daily sales is zero, days_until_stockout should be null rather than a fake number.
- Inactive products and inactive warehouses should be excluded from alert generation.
- Database failures should be logged server-side while returning a generic 500 status code to clients.

What would strengthen this part further

- **Pagination**: If a company has thousands of low-stock products, returning them all at once will be slow. Cursor-based pagination would be the right call here.
- **Caching**: This query is read-heavy and the data doesn't change by the second. A 5-minute Redis cache per company_id would take a lot of pressure off the DB.
- **Configurable time window**: Right now RECENT_SALES_WINDOW_DAYS is hardcoded. It could be a query param (?window=7) or a company-level setting.
- **Unit tests**: The SQL logic and the response shaping are both testable. I'd mock the DB pool and write tests against known fixture data.
- **Rate limiting**: Low-stock alerts might get polled frequently by a dashboard. Worth throttling to protect the DB.

THE API IS GIVEN IN Part3-api folder inside src/routes